Curso de Assimilação de Dados para Previsão Numérica de Tempo e Clima

Paulo Yoshio Kubota

Curso de Assimilação de Dados Atmosféricos

Carga horária sugerida: 40–60 horas

Formato: Aulas teóricas + exercícios práticos (Python/Fortran) + estudo de casos reais

Módulo 2 — Formulação matemática

1.Representação do estado

1. Vetor de estado **x**
2. Equações de evolução:

$$x_k = M_{k-1}(x_{k-1}) + \eta$$

1.Observações

1. Operador de observação **H**
2. Relação com o estado:

$$y_k = H_k(x_k) + \epsilon$$

1.Tipos de erros

1. Matriz de covariância de erro de background (**B**)
2. Matriz de covariância de erro de observação (**R**)

2.Função custo

1. Forma geral 3D-Var:

$$J(x) = \left(x - x_b\right)^T B^{-1}(x - x_b) + \left(y - Hx\right)^T R^{-1}(y - Hx)$$

Módulo 2 — Formulação Matemática

Paulo Yoshio Kubota

Módulo 2 — Formulação Matemática

1. Representação do Estado

O **estado do sistema** *é a descrição matemática* das **condições da atmosfera em um dado momento**.
O vetor $\vec{x}$ representa esse estado da **condições da atmosfera em um dado momento**, que contém variáveis como temperatura, umidade, pressão e vento em diversos pontos de uma grade tridimensional.

O **objetivo da assimilação de dados** é ajustar esse vetor $\vec{x}$ para que ele se aproxime o mais possível das observações reais y.

1.1 Definição do Vetor de Estado

O vetor de estado $\vec{x}$ pode ser representado como uma combinação de variáveis físicas na grade do modelo:

$$x = [T_1 \quad T_2 \quad T_3 \quad \dots \quad T_n]^T$$

onde T_i é a variável física no ponto i da grade (por exemplo, temperatura).

Assim, $\vec{x}$ pode ser um vetor de temperatura, umidade, ou uma combinação dessas variáveis.

Módulo 2 — Formulação Matemática

1. Representação do Estado

1.2 Equações de Evolução

A **equação de evolução do estado** é dada pela equação de previsão:

$$x_k = \boldsymbol{M_{k-1}(x_{k-1})} + \eta$$

onde:

- x_k é o vetor de estado no passo k,
- M_{k-1} é o **operador do modelo** (pode ser uma equação diferencial),
- η é o erro ou **ruído no modelo**, que representa incertezas do modelo.

Módulo 2 — Formulação Matemática

2. Observações

As **observações** do sistema, **definida por y,** são feitas em pontos específicos da grade ou em variáveis específicas (como a temperatura na superfície, ou umidade em uma estação).

2.1 Operador de Observação (H)

O **operador de observação H mapeia o vetor de estado para as observações**. Ele pode ser linear ou não linear, dependendo do tipo de dados observados. Em um modelo simples, temos:

$$y_k = \boldsymbol{H_k(x_k)} + \epsilon$$

onde:

y_k é o vetor de observações no passo k,

$\boldsymbol{H_k(x_k)}$ é a previsão do modelo para as observações,

$\boldsymbol{\epsilon}$ é o erro ou ruído da observação, que representa incertezas nos dados.

Módulo 2 — Formulação Matemática

2. Observações

As **observações** do sistema, **definida por y,** são feitas em pontos específicos da grade ou em variáveis específicas (como a temperatura na superfície, ou umidade em uma estação).

2.2 Erro de Observação (R(ϵ))

A matriz **R** é a **matriz de covariância do erro de observação**. Ela descreve a incerteza ϵ associada a **cada observação** e é crucial para o ajuste do modelo.

$$y_k = H_k(x_k) + \epsilon$$

Função Custo

A função de custo não é “descoberta”: ela é **construída** a partir da Premissa 2 e depois usada para achar os pesos A_i.

$$T_a = T_b + \frac{\sigma_b^2}{\sigma_b^2 + \sigma_o^2}(T_o - T_b)$$

$$\mathbf{x}_a = \mathbf{x}_b + \mathbf{K}\left[\mathbf{y}_o - H(\mathbf{x}_b)\right]$$

1. Definição (Premissa 2)

Queremos que a análise z_a fique o mais próxima possível do valor verdadeiro z^t. Medimos isso pelo erro quadrático médio:

$$J = \frac{1}{2}\langle (z^t - z_a)^2 \rangle$$

Os colchetes $\langle\ \rangle$ são a média sobre muitos casos. O $\frac{1}{2}$ é só conveniência para a derivada.

Função Custo

2. Substituição (Premissa 1)

Trocando z_a pela média ponderada da Premissa 1, obtemos a expressão do slide:

$$J = \left\langle \frac{1}{2}\left[z^t - z_b^{fg} - \sum_{i=1}^{m} A_i\left(z_i^{obs} - z_i^{fg}\right)\right]^2 \right\rangle$$

$$T_a = T_b + \frac{\sigma_b^2}{\sigma_b^2 + \sigma_o^2}(T_o - T_b)$$

$$\mathbf{x}_a = \mathbf{x}_b + \mathbf{K}\left[\mathbf{y}_o - H\left(\mathbf{x}_b\right)\right]$$

Agora J depende apenas dos pesos A_i. Eles são as incógnitas.

$$z_a = z_b^{fg} + \frac{{\sigma_i^b}^2}{{\sigma_i^b}^2 + {\sigma_i^o}^2}\left(z_i^{obs} - z_i^{fg}\right)$$

3. Como os pesos saem de J

Escreva tudo em termos de erros e suponha que os erros das observações são não viesados e não correlacionados com os do estado de referência:

$$J = \frac{1}{2}\langle (z^t - z_a)^2 \rangle$$

$\varepsilon_b = z_b^{fg} - z^t$ o erro do first guess no ponto de grade

$\varepsilon_i^b = z_i^{fg} - z_i^t$ o erro do first guess no local da observação i

$\varepsilon_i^o = z_i^{obs} - z_i^t$ o erro da observação i

$$z_a - z^t = \varepsilon_b + \sum_{i=1}^{m} A_i\left(\varepsilon_i^o - \varepsilon_i^b\right)$$ e J fica:

Função Custo

$$J = \left\langle \frac{1}{2}\left[z^t - z_b^{fg} - \sum_{i=1}^{m} A_i\left(z_i^{obs} - z_i^{fg}\right)\right]^2 \right\rangle$$

$$z_a - z^t = \varepsilon_b + \sum_{i=1}^{m} A_i\left(\varepsilon_i^o - \varepsilon_i^b\right)$$ e J fica:

$$J = \frac{1}{2}\langle (z^t - z_a)^2 \rangle$$

$$J = \frac{1}{2}\left\langle \left(-\varepsilon_b - \sum_{i=1}^{m} A_i\left(\varepsilon_i^o - \varepsilon_i^b\right)\right)^2 \right\rangle$$

$$J = \frac{1}{2}\left\langle (-\varepsilon_b)(-\varepsilon_b) - 2\varepsilon_b \sum_{i=1}^{m} A_i\left(\varepsilon_i^o - \varepsilon_i^b\right) + \left(\sum_{i=1}^{m} A_i\left(\varepsilon_i^o - \varepsilon_i^b\right)\right)^2 \right\rangle$$

$$J = \frac{1}{2}\left\langle \varepsilon_b\varepsilon_b - 2\varepsilon_b \sum_{i=1}^{m} A_i\left(\varepsilon_i^o - \varepsilon_i^b\right) + \sum_{i=1}^{m} A_i A_i\left(\varepsilon_i^o\varepsilon_i^o - 2\varepsilon_i^o\varepsilon_i^b + \varepsilon_i^b\varepsilon_i^b\right)\right\rangle$$

Função Custo

$$J = \frac{1}{2}\left\langle \varepsilon_b \varepsilon_b - 2\varepsilon_b \sum_{i=1}^{m} A_i\left(\varepsilon_i^o - \varepsilon_i^b\right) + \sum_{i=1}^{m} A_i A_i\left(\varepsilon_i^o \varepsilon_i^o - 2\varepsilon_i^o \varepsilon_i^b + \varepsilon_i^b \varepsilon_i^b\right)\right\rangle$$

$$J = \frac{1}{2}\left\langle \varepsilon_b \varepsilon_b - 2\sum_{i=1}^{m} A_i\left(\varepsilon_b \varepsilon_i^o - \varepsilon_b \varepsilon_i^b\right) + \sum_{i=1}^{m} A_i A_i\left(\varepsilon_i^o \varepsilon_i^o - 2\varepsilon_i^o \varepsilon_i^b + \varepsilon_i^b \varepsilon_i^b\right)\right\rangle$$

$\overline{\varepsilon_1} = \overline{\varepsilon_2} = 0$ Erros não tendenciosos

$\overline{\varepsilon_1^2} = \sigma_1^2 \quad \overline{\varepsilon_2^2} = \sigma_2^2$ Variâncias

$\overline{\varepsilon_1 \varepsilon_2} = \overline{(T_1 - T_t)}\,\overline{(T_2 - T_t)} = 0$ Não correlacionados

$$J = \frac{1}{2}\left\langle \overline{\varepsilon_b \varepsilon_b} - 2\sum_{i=1}^{m} A_i\left(\overline{\varepsilon_b \varepsilon_i^o} - \overline{\varepsilon_b \varepsilon_i^b}\right) + \sum_{i=1}^{m} A_i A_i\left(\overline{\varepsilon_i^o \varepsilon_i^o} - 2\overline{\varepsilon_i^o \varepsilon_i^b} + \overline{\varepsilon_i^b \varepsilon_i^b}\right)\right\rangle$$

$$J = \frac{1}{2}\left\langle \langle \varepsilon_b{}^2 \rangle + 2\sum_{i=1}^{m} A_i \langle \varepsilon_b \varepsilon_i^b \rangle + \sum_{i=1}^{m} A_i A_i\left(\langle \varepsilon_i^o \varepsilon_i^o \rangle + \langle \varepsilon_i^b \varepsilon_i^b \rangle\right)\right\rangle$$

Função Custo

$$J = \frac{1}{2}\left\langle \langle {\varepsilon_b}^2 \rangle + 2\sum_{i=1}^{m} A_i \langle \varepsilon_b \varepsilon_i^b \rangle + \sum_{i=1}^{m} A_i A_i \left(\langle \varepsilon_i^o \varepsilon_i^o \rangle + \langle \varepsilon_i^b \varepsilon_i^b \rangle \right) \right\rangle$$

Aqui:

- $B_{ij} = \langle \varepsilon_i^b \varepsilon_i^b \rangle$ é a covariância dos erros do first guess entre os pontos de observação.
- $R_{ij} = \langle \varepsilon_i^o \varepsilon_i^o \rangle$ é a covariância dos erros das observações.
- $b_i = \langle \varepsilon_b \varepsilon_i^b \rangle$ é a covariância entre o erro do first guess no ponto de grade e no ponto da observação.

Minimizar J significa impor ∂J/∂A_k = 0 para cada k:

$$J = \frac{1}{2}\left\langle \langle {\varepsilon_b}^2 \rangle + 2\sum_{i=1}^{m} A_i b_i + \sum_{i=1}^{m} A_i A_i \left(R_{ij} + B_{ij} \right) \right\rangle$$

Função Custo

$$J = \frac{1}{2}\left\langle \langle {\varepsilon_b}^2 \rangle + 2\sum_{i=1}^{m} A_i b_i + \sum_{i=1}^{m} A_i A_i \left(R_{ij} + B_{ij}\right) \right\rangle$$

Minimizar J significa impor $\frac{\partial J}{\partial A_k}$= 0 para cada k:

$$\frac{\partial J}{\partial A_k} = \frac{1}{2}\left\langle \frac{\partial}{\partial A_k}\langle {\varepsilon_b}^2 \rangle + \frac{\partial}{\partial A_k} 2\sum_{i=1}^{m} A_i b_i + \frac{\partial}{\partial A_k}\sum_{i=1}^{m} A_i A_i \left(R_{ij} + B_{ij}\right) \right\rangle$$

$$\frac{\partial J}{\partial A_k} = \frac{1}{2}\left\langle 2\sum_{i=1}^{m} \frac{\partial A_i}{\partial A_k} b_i + \sum_{i=1}^{m} \frac{\partial A_i}{\partial A_k} A_i \left(R_{ij} + B_{ij}\right) + \sum_{i=1}^{m} A_i \frac{\partial A_i}{\partial A_k} \left(R_{ij} + B_{ij}\right) \right\rangle$$

$i = k$

$$\frac{\partial J}{\partial A_k} = \frac{1}{2}\left\langle 2\sum_{k=1}^{m} \frac{\partial A_k}{\partial A_k} b_k + \sum_{k=1}^{m} \frac{\partial A_k}{\partial A_k} A_k \left(R_{kj} + B_{kj}\right) + \sum_{k=1}^{m} A_k \frac{\partial A_k}{\partial A_k} \left(R_{kj} + B_{kj}\right) \right\rangle$$

$$\frac{\partial J}{\partial A_k} = \frac{1}{2}\left\langle 2\sum_{k=1}^{m} b_k + 2\sum_{k=1}^{m} A_k \left(R_{kj} + B_{kj}\right) \right\rangle$$

Função Custo

$$\frac{\partial J}{\partial A_k} = \frac{1}{2}\left\langle 2\sum_{k=1}^{m} b_k + 2\sum_{k=1}^{m} A_k\left(R_{kj} + B_{kj}\right)\right\rangle = 0$$

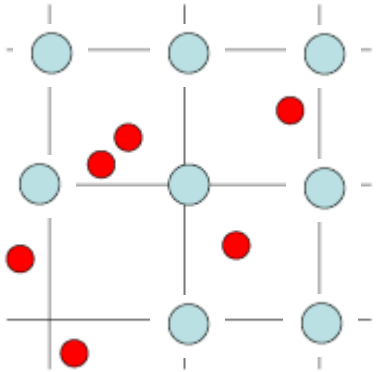
$$\sum_{k=1}^{m} b_k + \sum_{k=1}^{m} A_k\left(R_{kj} + B_{kj}\right)$$

Minimizar J significa impor $\frac{\partial J}{\partial A_k} = 0$ para cada k:

$$\sum_{k=1}^{m} A_k\left(R_{kj} + B_{kj}\right) + b_k = 0 \qquad (B + R)A = b \qquad A = (B + R)^T b$$

Esse é o sistema de equações normais da **interpolação ótima**.
Os pesos ótimos dependem das covariâncias dos erros: observações precisas (R pequeno) e próximas (b grande) recebem mais peso.
Como os pontos de observação e a grade mudam a cada caso, os pesos são escolhidos para minimizar o erro médio **ao longo de muitos casos**, e não para uma análise isolada.

Função Custo



- Observações de altura geopotencial (z_i^{obs}, z_i^{fg})
- Ponto de grade (z_a, z_b^{fg}) , onde $z_a = analise$

Aqui, z_b^{fg} se refere à **primeira estimativa (ou estado de referência)**, que é derivada de uma **previsão de curto prazo anterior**.

Premissa 1: A **análise** é definida como uma **média ponderada sobre as observações vizinhas:**

$$z_a = z_b^{fg} + \sum_{i=1}^{m} A_i\left(z_i^{obs} - z_i^{fg}\right)$$

Aqui, z_i^{fg} é a **interpolação de z_b^{fg} para a localização da observação**.

Premissa 2: Os **pesos** A_i são escolhidos de forma que o **erro médio ao longo de muitos casos seja minimizado** (no sentido de **mínimos quadrados**). Para isso, o seguinte funcional deve ser minimizado:

$$J = \frac{1}{2}\langle (z^t - z_a)^2 \rangle = \left\langle \frac{1}{2}\left[z^t - z^{fg} - \sum_{i=1}^{m} A_i\left(z_i^{obs} - z_i^{fg}\right)\right]^2 \right\rangle$$

onde z^t define o **valor verdadeiro no ponto de grade**, e
onde os **colchetes externos** indicam a **média ao longo de algum período de tempo**.

Função Custo

A função de custo do 3D-Var. Ela pode ser obtida de duas formas: a mais comum parte do teorema de Bayes com erros gaussianos, e a outra mostra a equivalência com a interpolação ótima do slide anterior.

1. Via Bayes (máxima probabilidade a posteriori)

Queremos o estado x mais provável dado as observações y:

$$P(x|y) \propto P(x|y)P(x)$$

A priori $P(x)$: o estado de referência x_b com erro $\varepsilon_b = x_b - x^t$, gaussiano, $\mu = 0$ de média zero e covariância B:

A priori é uma “nuvem de probabilidade” centrada em x_b . O estado verdadeiro x^t cai em algum lugar dentro dela, e a seta $\varepsilon_b = x_b - x^t$ é o erro do first guess.

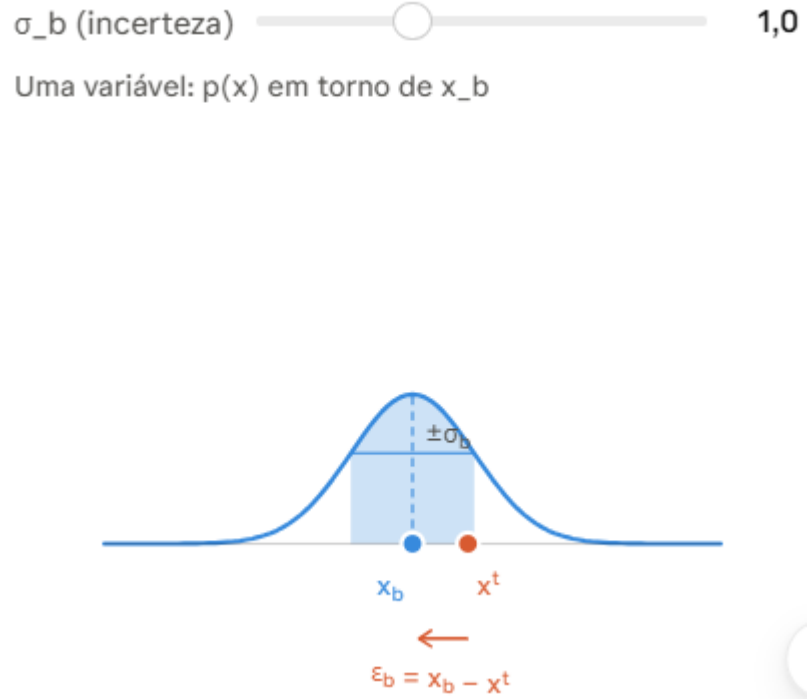
Função Custo

À esquerda (uma variável)

•A curva é $P(x) \propto e^{-\left[\frac{1(x-x_b)^2}{2 \quad \sigma_b{}^2}\right]}$

•A faixa sombreada vai de $-\sigma_b$ a $+\sigma_b$ e concentra cerca de 68% da probabilidade.

•Aumentar σ_b alarga a curva, o que significa menos confiança no first guess. É o termo que faz B^{-1} pesar menos em J.



Função Custo

À direita (duas variáveis)

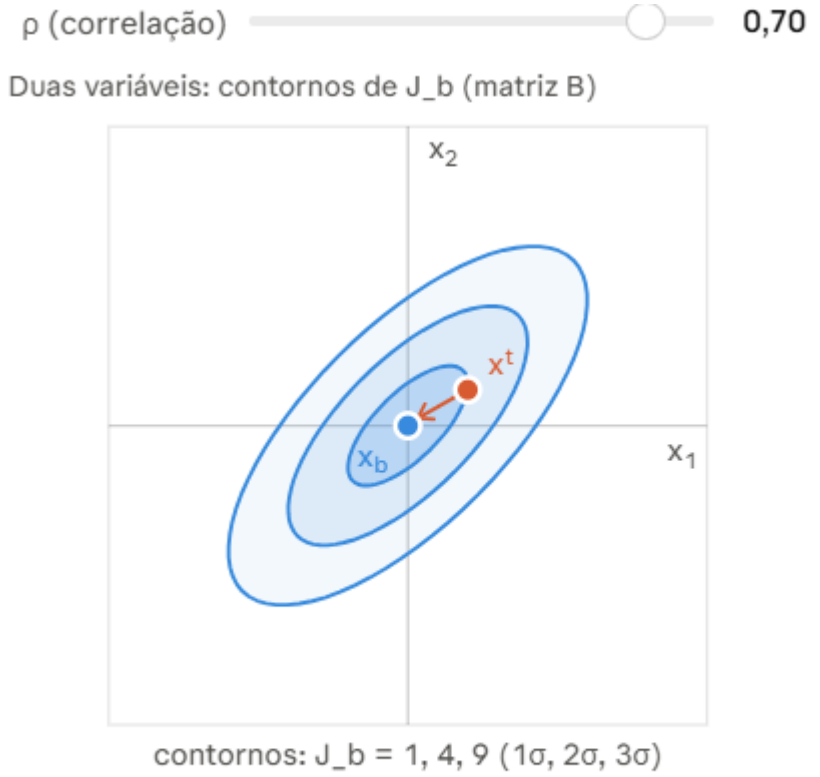
•A covariância B vira elipses.

$$J_b(x) = (x - x_b)^T B^{-1}(x - x_b)$$

Cada elipse é um contorno de constante, com valores 1, 4 e 9 para 1σ, 2σ e 3σ.

•Com $\rho = 0$, as elipses são círculos e os erros dos dois pontos de grade são independentes.

•Com $\rho > 0$, a elipse se alonga ao longo da diagonal. Erros correlacionados (os dois pontos errados no mesmo sentido) saem "baratos" em J_b. Já um erro na direção perpendicular, com sinais opostos, é muito penalizado.



Função Custo

A priori p(x): o estado de referência x_b com erro $\varepsilon_b = x_b - x^t$, gaussiano, de média zero e covariância B:

- $P(x) \propto e^{-\left[\frac{1}{2}(x-x_b)^T B^{-1}(x-x_b)\right]}$

•é $P(x) \propto e^{-\left[\frac{1(x-x_b)^2}{2 \quad \sigma_b{}^2}\right]}$

Verossimilhança $\mathrm{P}(y|x)$: as observações se relacionam com o estado por $y = H(x) + \varepsilon_o$, com ε_o gaussiano, de média zero e covariância R. H é o operador de observação, que leva o espaço do modelo ao espaço das observações:

$$\mathrm{P}(y|x) \propto e^{-\left[\frac{1}{2}(y-Hx)^T R^{-1}(y-Hx)\right]}$$

Multiplicando as duas, os expoentes se somam:

$$P(x|y) \propto P(x|y)P(x)$$

$$P(x|y) \propto e^{-\left[\frac{1}{2}(x-x_b)^T B^{-1}(x-x_b)\right]} e^{-\left[\frac{1}{2}(y-Hx)^T R^{-1}(y-Hx)\right]}$$

Função Custo

$$P(x|y) \propto e^{-\left[\frac{1}{2}(x-x_b)^T B^{-1}(x-x_b)\right]} e^{-\left[\frac{1}{2}(y-Hx)^T R^{-1}(y-Hx)\right]}$$

Maximizar $p(x|y)$ é o mesmo que minimizar o argumento da exponencial. Chamando esse argumento de J (o fator ½ é só convenção e não muda o mínimo), chegamos a:

$$J(x) = (x - x_b)^T B^{-1}(x - x_b) + (y - Hx)^T R^{-1}(y - Hx)$$

O termo em azul é o desvio em relação ao first guess, ponderado pela confiança nele (B^{-1}). O outro é o desvio em relação às observações, ponderado por R^{-1}.

Módulo 2 — Formulação Matemática

3. Função Custo

A **função custo** é uma ***medida da discrepância entre o estado do modelo e as observações***. O objetivo da assimilação de dados é ***minimizar essa função para obter o melhor estado possível do sistema***.

3.1 Função Custo para 3D-Var

A forma clássica da função custo no método **3D-Var** é dada por:

$$J(x) = (x - x_b)^T B^{-1}(x - x_b) + (y - Hx)^T R^{-1}(y - Hx)$$

$$x_k = \boldsymbol{M_{k-1}(x_{k-1})} + \eta$$

$$y_k = \boldsymbol{H_k(x_k)} + \epsilon$$

$$\eta = \boldsymbol{M_{k-1}(x_{k-1})} - x_k$$

$$\epsilon = \boldsymbol{H_k(x_k)} - y_k$$

onde:

x_b é o estado de fundo (modelo),

B é a matriz de covariância do erro de fundo (modelo),

y são as observações,

H é o operador de observação,

R é a matriz de covariância do erro da observação.

Módulo 2 — Formulação Matemática

3. Função Custo

$$J(x) = (x - x_b)^T B^{-1}(x - x_b) + (y - Hx)^T R^{-1}(y - Hx)$$

3.2 Significado dos Termos

- O primeiro termo $(x - x_b)^T B^{-1}(x - x_b)$ mede a **discrepância entre** o estado do modelo e o estado de fundo, ajustado pelas incertezas do modelo.

- O segundo termo $(y - Hx)^T R^{-1}(y - Hx)$ mede a **discrepância entre** as observações e as previsões do modelo, **ajustadas pelas incertezas H das observações**.

Módulo 2 — Formulação Matemática

4. Método de Mínimos Quadrados

Uma maneira de minimizar a função custo é através de um **método de mínimos quadrados**.

O objetivo é encontrar o valor de x que minimiza a função $J(x)$.

Módulo 2 — Formulação Matemática

4.1 Derivação e Solução

A solução para o problema de otimização é dada por:

$$\hat{x} = x_b + BH^T\left(HBH^T + R\right)^{-1}(y - Hx_b)$$

onde:

$\hat{x}$ é o estado corrigido após a assimilação,
B é a matriz de covariância do erro do modelo,
H é o operador de observação,
R é a matriz de covariância do erro da observação.

Esta fórmula mostra como o estado x do modelo é ajustado pelas observações y, **ponderado pelas incertezas** no modelo (B) e das observações (R).

2) Gradiente de $J(x)$

Calcule a derivada de $J(x)$ em relação a x. Usando regras de derivação matricial (e lembrando que $\nabla_x(x^T Ax) = (A + A^T)x$; aqui B^{-1}e R^{-1}são simétricos:

$$J(x) = (x - x_b)^T B^{-1}(x - x_b) + (y - Hx)^T R^{-1}(y - Hx)$$

Para o primeiro termo $(x - x_b)^T B^{-1}(x - x_b)$:

$$\nabla_x[(x - x_b)^T B^{-1}(x - x_b)] = [(1)^T B^{-1}(x - x_b) + (x - x_b)^T B^{-1}(1)] = 2B^{-1}(x - x_b)$$

$$\nabla_x[(x - x_b)^T B^{-1}(x - x_b)] = [B^{-1}(x - x_b) + B^{-1}(x - x_b)^T] = 2B^{-1}(\boldsymbol{x - x_b})$$

Para o segundo termo (note $\boldsymbol{y - Hx}$):

$$\nabla_x[(y - Hx)^T R^{-1}(y - Hx)] = [(-H)^T B^{-1}(y - Hx) + (y - Hx)^T B^{-1}(-H)] = 2H^T R^{-1}(y - Hx)$$

$$\nabla_x[(y - Hx)^T R^{-1}(y - Hx)] = [-(H)^T R^{-1}(\boldsymbol{y - Hx}) - (H)R^{-1}(\boldsymbol{y - Hx})^T] = 2H^T R^{-1}(y - Hx)$$

Somando:

$$\nabla_x J(x) = 2[B^{-1}(x - x_b) - H^T R^{-1}(y - Hx)]$$

3) Condição de ótimo (gradiente zero)

$$\nabla_x J(x) = 0$$

$$2[B^{-1}(x - x_b) - H^T R^{-1}(y - Hx)] = 0$$

$$[B^{-1}(x - x_b) - H^T R^{-1}(y - Hx)] = 0$$

Rearranjando:

$$[(B^{-1}x - B^{-1}x_b) - (H^T R^{-1}y - H^T R^{-1}Hx)] = 0$$

$$B^{-1}x - B^{-1}x_b \; - H^T R^{-1}y + H^T R^{-1}Hx = 0$$

Agrupando termos em x:

$$B^{-1}x + H^T R^{-1}Hx = B^{-1}x_b + H^T R^{-1}y$$

$$(B^{-1} + H^T R^{-1}H)x = B^{-1}x_b + H^T R^{-1}y$$

Essa é a **equação normal** do problema. A matriz do lado esquerdo é simétrica definida positiva (soma de SPD), logo a solução é única.

4) Solução explícita (forma direta)

$$(B^{-1} + H^T R^{-1} H)x = B^{-1}x_b + H^T R^{-1} y$$

$$(B^{-1} + H^T R^{-1} H)x = (B^{-1}x_b + H^T R^{-1} y)$$

$$x = (B^{-1} + H^T R^{-1} H)^{-1}(B^{-1}x_b + H^T R^{-1} y)$$

Uma forma equivalente, mais usada em assimilação (forma do ganho — "análise incremental"):

Objetivo

Mostrar que

$$x_a = \left(B^{-1} + H^T R^{-1} H\right)^{-1}\left(B^{-1}x_b + H^T R^{-1} y\right)$$

é equivalente a

$$x_a = x_b + BH^T\left(HBH^T + R\right)^{-1}(y - Hx_b).$$

Defina $S = HBH^T + R$ (matriz de inovação) — é invertível se R e B forem SPD.

Passo 1 — identidade de Woodbury

$$x = (B^{-1} + H^T R^{-1} H)^{-1}(B^{-1} x_b + H^T R^{-1} y)$$

A identidade de Woodbury (forma útil aqui) é, para matrizes conformes:

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}$$

Escolha

$$A = B^{-1}; U = H^T; C = R^{-1}; V = H$$

Então

$$A + UCV = B^{-1} + H^T R^{-1} H$$

Note que usamos

$$(C^{-1} + VA^{-1}U)^{-1} = (R + HBH^T)^{-1} \qquad e \qquad A^{-1} = B$$

e a identidade dá

$$(B^{-1} + H^T R^{-1} H)^{-1} = B - BH^T(R + H^T BH)^{-1}HB$$

Passo 1 — identidade de Woodbury

$$(B^{-1} + H^T R^{-1} H)^{-1} = B - BH^T (R + H^T BH)^{-1} HB$$

Chamando (a Matriz inovação)

$$S = HBH^T + R$$

podemos escrever que:

$$M \equiv (B^{-1} + H^T R^{-1} H)^{-1} = B - BH^T (S)^{-1} HB$$

Passo 2 — multiplique M por os termos da rhs

Queremos aplicar M ao vetor $X_a = B^{-1}x_b + H^T R^{-1}y$.**Use linearidade $f(x_a) = w_1 x_b + w_2 y$**:

$$X_a = B^{-1}x_b + H^T R^{-1}y$$

$$MX_a = MB^{-1}x_b + MH^T R^{-1}y$$

$$x_a = MX_a = M(B^{-1}x_b) + M(H^T R^{-1}y)$$

$$x_a = M(B^{-1}x_b) + M(H^T R^{-1}y)$$

Usando (1):

Termo A: $MB^{-1}x_b$

Termo B: $MH^T B^{-1}$y

Passo 2 — multiplique M por os termos da rhs $x_a = M(B^{-1}x_b) + M(H^T R^{-1} y)$

Calcule cada termo usando (1).

Termo A: $MB^{-1}x_b$ Usando (1):

$$M \equiv (B^{-1} + H^T R^{-1} H)^{-1} = B - BH^T (S)^{-1} HB$$

$$MB^{-1} = (B - BH^T (S)^{-1} HB)B^{-1} = (BB^{-1} - BH^T (S)^{-1} HBB^{-1})$$

$$MB^{-1} = (I - BH^T (S)^{-1} HI)$$

$$MB^{-1} = (1 - BH^T (S)^{-1} H)$$

$$MB^{-1}x_b = (1 - BH^T (S)^{-1} H)x_b$$

$$MB^{-1}x_b = x_b - (BH^T (S)^{-1} H)x_b$$

Passo 2 — multiplique M por os termos da rhs $x_a = M(B^{-1}x_b) + M(H^T R^{-1} y)$

Calcule cada termo usando (1).

Termo B: $MH^T B^{-1}$y

Usando (1) novamente:

$$MH^T R^{-1} = (B - BH^T S^{-1} HB) H^T R^{-1}$$

$$MH^T R^{-1} = (B - BS^{-1} HBH^T) H^T R^{-1}$$

$$MH^T R^{-1} = \left(B - BS^{-1}(S - R)\right) H^T R^{-1}$$

Reorganize observando que $HBH^T = S - R$

$$MH^T R^{-1} = (B - BS^{-1}S + BS^{-1}R) H^T R^{-1}$$

$$MH^T R^{-1} = (B - B + BS^{-1}R) H^T R^{-1}$$

$$MH^T R^{-1} = (BS^{-1}RR^{-1}) H^T$$

$$MH^T R^{-1} = (BH^T S^{-1})$$

$$MH^T R^{-1} y = (BH^T S^{-1}) y$$

Passo 2 — multiplique M por os termos da rhs $x_a = M(B^{-1}x_b) + M(H^T R^{-1} y)$

Calcule cada termo usando (1).

Termo B: $MH^T B^{-1}$y

Usando (1) novamente:

$$MH^T R^{-1} = (B - BH^T S^{-1} HB) H^T R^{-1}$$

$$MH^T R^{-1} = (B - BS^{-1} HBH^T) H^T R^{-1}$$

$$MH^T R^{-1} = \left(B - BS^{-1}(S - R)\right) H^T R^{-1}$$

Reorganize observando que $HBH^T = S - R$

$$MH^T R^{-1} = (B - BS^{-1}S + BS^{-1}R) H^T R^{-1}$$

$$MH^T R^{-1} = (B - B + BS^{-1}R) H^T R^{-1}$$

$$MH^T R^{-1} = (BS^{-1}RR^{-1}) H^T$$

$$MH^T R^{-1} = (BH^T S^{-1})$$

$$MH^T R^{-1} y = (BH^T S^{-1}) y$$

Passo 3 — somar Termo A=$MB^{-1}x_b = x_b - (BH^T(S)^{-1}H)x_b$**+ Termo B=**$MH^TR^{-1}y = (BH^TS^{-1})y$

$$x_a = M(B^{-1}x_b) + M(H^TR^{-1}y)$$

$$x_a = x_b - (BH^T(S)^{-1})Hx_b + (BH^TS^{-1})y$$

Agrupar

$$x_a = x_b + (BH^TS^{-1})[-Hx_b + y]$$

$$S = HBH^T + R$$

$$x_a = x_b + (HBH^T + R)^{-1}BH^T[y - Hx_b]$$

E esta é exatamente a **forma do ganho**:

$$x_a = x_b + BH^T(HBH^T + R)^{-1}[y - Hx_b]$$

como queríamos demonstrar.

Observação (verificação da identidade $MH^T R^{-1} = BH^T S^{-1}$)

Para quem quiser ver esse passo em detalhe, multiplique:

$$MH^T R^{-1} \overset{(1)}{=} (B - BH^T S^{-1} HB) H^T R^{-1}.$$
$$MH^T R^{-1} \overset{(1)}{=} BH^T R^{-1} - BH^T S^{-1} HBH^T R^{-1}.$$
$$MH^T R^{-1} \overset{(1)}{=} BH^T R^{-1}[1 - S^{-1} HBH^T].$$

Agora note que $S = HBH^T + R \Rightarrow R = S - HBH^T$.=> $HBH^T = S - \mathrm{R}$.

$$MH^T R^{-1} \overset{(1)}{=} BH^T R^{-1}[1 - S^{-1} HBH^T].$$
$$MH^T R^{-1} \overset{(1)}{=} BH^T R^{-1}[1 - S^{-1}(S - R)].$$
$$MH^T R^{-1} \overset{(1)}{=} BH^T R^{-1}[1 - S^{-1} S + S^{-1} R].$$
$$MH^T R^{-1} \overset{(1)}{=} BH^T R^{-1}[1 - 1 + S^{-1} R].$$
$$MH^T R^{-1} \overset{(1)}{=} BH^T R^{-1}[S^{-1} R].$$
$$MH^T R^{-1} \overset{(1)}{=} BH^T R^{-1} R[S^{-1}].$$
$$MH^T R^{-1} \overset{(1)}{=} BH^T [S^{-1}].$$
$$MH^T R^{-1} \overset{(1)}{=} BH^T S^{-1}$$

Observação (verificação da identidade $MH^T R^{-1} = BH^T S^{-1}$)

$$(BH^T S^{-1})S = BH^T.$$

E **comparando com a expressão acima e reordenando** termos (uso de $S = HBH^T + R$ (chega-se à igualdade.

Em outras palavras, **ambos os lados são soluções da mesma equação linear** e, por identidade de Woodbury, são iguais.

(A demonstração formal passa por manipular e cancelar os mesmos termos BH^T e $BH^T S^{-1} HBH^T$(.

Verificação simbólica com SymPy (equivalência algébrica)

```
import sympy as sp

# Definir dimensões genéricas
n, m = sp.symbols('n m', integer=True, positive=True)

# Definir matrizes simbólicas
B = sp.MatrixSymbol('B', n,  n)
R = sp.MatrixSymbol('R', m, m)
H = sp.MatrixSymbol('H', m, n)
xb = sp.MatrixSymbol('x_b', n, 1)
y = sp.MatrixSymbol('y', m, 1)

# Expressões simbólicas
B_inv = sp.Inverse(B)
R_inv = sp.Inverse(R)

# Forma 1 (normal)
xa1 = sp.Inverse(B_inv + H.T * R_inv * H) * (B_inv * xb + H.T * R_inv * y)

# Forma 2 (ganho)
xa2 = xb + B * H.T * sp.Inverse(H * B * H.T + R) * (y - H * xb)

# Simplificar diferença
diff = sp.simplify(xa1 - xa2)
sp.pretty_print(diff)
```

Objetivo

Mostrar que

$$x_a = \left(B^{-1} + H^T R^{-1} H\right)^{-1} \left(B^{-1} x_b + H^T R^{-1} y\right)$$

é equivalente a

$$x_a = x_b + BH^T \left(HBH^T + R\right)^{-1} (y - Hx_b).$$

Defina $S = HBH^T + R$ (matriz de inovação) — é invertível se R e B forem SPD.

Matrix([[0], [0], ..., [0]])

$$M \equiv (B^{-1} + H^T R^{-1} H)^{-1} = B - BH^T (S)^{-1} HB$$

Vamos usar números concretos (dimensões pequenas) e calcular x_a pelas **duas fórmulas**.

```
import numpy as np

# Dados exemplo
B = np.array([[1.0, 0.2], [0.2, 2.0]])
R = np.array([[0.5]])
H = np.array([[1.0, 0.0]])
xb = np.array([[10.0], [12.0]])
y  = np.array([[11.0]])

# --- Forma 1 (normal) ---
B_inv = np.linalg.inv(B)
R_inv = np.linalg.inv(R)
A = B_inv + H.T @ R_inv @ H
rhs = B_inv @ xb + H.T @ R_inv @ y
xa1 = np.linalg.solve(A, rhs)

# --- Forma 2 (ganho) ---
S = H @ B @ H.T + R
K = B @ H.T @ np.linalg.inv(S)
xa2 = xb + K @ (y - H @ xb)

# --- Comparação ---
print("xa (forma normal):\n", xa1)
print("xa (forma ganho):\n", xa2)
print("Diferença:", np.max(np.abs(xa1 - xa2)))
```

Objetivo

Mostrar que

A

$$x_a = \left(B^{-1} + H^T R^{-1} H\right)^{-1} \left(B^{-1} x_b + H^T R^{-1} y\right)$$

é equivalente a

K

$$x_a = x_b + BH^T \left(HBH^T + R\right)^{-1} (y - Hx_b).$$

Defina $S = HBH^T + R$ (matriz de inovação) — é invertível se R e B forem SPD.

```
xa (forma normal):
 [[10.6667]
 [12.1333]]
xa (forma ganho):
 [[10.6667]
 [12.1333]]
Diferença: ~1e-15
```



Vamos usar números concretos (dimensões pequenas) e calcular x_a pelas **duas fórmulas**.



```
program var3d_equivalence
   implicit none
   integer, parameter :: n = 2, m = 1
   real :: B(n,n)    , R(m,m)    , H(m,n), xb(n),  y(m)
   real :: Binv(n,n), Rinv(m,m), A(n,n)  ,  rhs(n)
   real :: xa1(n)    , xa2(n)       , S(m,m), K(n,m)
   integer :: i, j
   ! Dados
   B = reshape([1.0, 0.2, 0.2, 2.0], [n,n])
   R = reshape([0.5], [m,m])
   H = reshape([1.0, 0.0], [m,n])
   xb = [10.0, 12.0]
   y = [11.0]

   ! Inversas (2x2 e 1x1 simples)
   Binv(1,1) =  B(2,2)/(B(1,1)*B(2,2)-B(1,2)*B(2,1))
   Binv(2,2) =  B(1,1)/(B(1,1)*B(2,2)-B(1,2)*B(2,1))
   Binv(1,2) = -B(1,2)/(B(1,1)*B(2,2)-B(1,2)*B(2,1))
   Binv(2,1) = Binv(1,2)
   Rinv(1,1) = 1.0/R(1,1)

   ! ---- Forma 1 ----
   A    = Binv + matmul(transpose(H), matmul(Rinv, H))
   rhs = matmul(Binv, xb) + matmul(transpose(H), matmul(Rinv, y))
   xa1 = matmul(inverse(A), rhs)

   ! ---- Forma 2 ----
   S = matmul(H, matmul(B, transpose(H))) + R
   K = matmul(B, matmul(transpose(H), inverse(S)))
   xa2 = xb + matmul(K, (y - matmul(H, xb)))

   print *, "xa1 (normal): ", xa1
   print *, "xa2 (ganho):  ", xa2
   print *, "Diferença: ", maxval(abs(xa1 - xa2))
end program var3d_equivalence
```

Use `contains` + função `inverse()` de matriz pequena (ou biblioteca LAPACK `dgesv` para versão geral).
Resultado: diferença numérica ~1e-15.

Forma	Expressão	Observação
Normal	$x_a = (B^{-1} + H^T R^{-1} H)^{-1}(B^{-1} x_b + H^T R^{-1} y)$	Operações no **espaço de estado**
Ganho (prática)	$x_a = x_b + BH^T(HBH^T + R)^{-1}(y - Hx_b)$	Operações no **espaço de observação**, mais eficiente

Ambas são matematicamente equivalentes — o **script simbólico confirma** a identidade e o **exemplo numérico** mostra que, mesmo com arredondamento, as duas produzem o mesmo resultado.

Vamos usar números concretos (dimensões pequenas) e calcular x_a pelas **duas fórmulas**.

```
program var3d_equivalencia
  implicit none
  real(8), dimension(3) :: xb, xa1, xa2, y
  real(8), dimension(3,3) :: B, Binv, Ht, I
  real(8), dimension(1,3) :: H
  real(8), dimension(1,1) :: R, tmp1, tmp2
  integer :: i

  xb = (/10.0d0, 12.0d0, 14.0d0/)
  y  = (/13.0d0/)

  B = 0.0d0
  do i=1,3
    B(i,i) = 1.0d0
  end do

  H = reshape([0.0d0, 1.0d0, 0.0d0], shape(H))
  Ht = transpose(H)
  R(1,1) = 0.5d0

  ! Binv = inv(B) (como B = I, é igual)
  Binv = B
  I = B

  ! --- Forma 1 (Kalman gain) ---
  tmp1 = matmul(H, matmul(B, Ht)) + R
  xa1 = xb + matmul(matmul(B, Ht), matmul(inv1x1(tmp1), (y - matmul(H, xb))))

  ! --- Forma 2 (minimização direta) ---
  xa2 = matmul(inv3x3(Binv + matmul(Ht, matmul(inv1x1(R), H))), &
       matmul(Binv, xb) + matmul(Ht, matmul(inv1x1(R), y)))

  print *, 'xa (forma Kalman):', xa1
  print *, 'xa (forma minimização):', xa2
  print *, 'diferença:', xa1 - xa2

contains

  function inv1x1(a) result(ainv)
    real(8), dimension(1,1), intent(in) :: a
    real(8), dimension(1,1) :: ainv
    ainv(1,1) = 1.0d0 / a(1,1)
  end function inv1x1

  function inv3x3(a) result(ainv)
    real(8), dimension(3,3), intent(in) :: a
    real(8), dimension(3,3) :: ainv
    real(8) :: det
    det = a(1,1)*(a(2,2)*a(3,3)-a(2,3)*a(3,2)) - a(1,2)*(a(2,1)*a(3,3)-a(2,3)*a(3,1)) + a(1,3)*(a(2,1)*a(3,2)-a(2,2)*a(3,1))
    ainv = transpose(reshape([ &
      (a(2,2)*a(3,3)-a(2,3)*a(3,2)), -(a(1,2)*a(3,3)-a(1,3)*a(3,2)),  (a(1,2)*a(2,3)-a(1,3)*a(2,2)), &
     -(a(2,1)*a(3,3)-a(2,3)*a(3,1)),  (a(1,1)*a(3,3)-a(1,3)*a(3,1)), -(a(1,1)*a(2,3)-a(1,3)*a(2,1)), &
      (a(2,1)*a(3,2)-a(2,2)*a(3,1)), -(a(1,1)*a(3,2)-a(1,2)*a(3,1)),  (a(1,1)*a(2,2)-a(1,2)*a(2,1))], [3,3])) / det
  end function inv3x3

end program var3d_equivalencia
```

Forma	Expressão	Observação
Normal	$x_a = (B^{-1} + H^T R^{-1} H)^{-1}(B^{-1} x_b + H^T R^{-1} y)$	Operações no **espaço de estado**
Ganho (prática)	$x_a = x_b + BH^T(HBH^T + R)^{-1}(y - Hx_b)$	Operações no **espaço de observação**, mais eficiente

Ambas são matematicamente equivalentes — o **script simbólico confirma** a identidade e o **exemplo numérico** mostra que, mesmo com arredondamento, as duas produzem o mesmo resultado.

Modelo físico: advecção–difusão 1D

A equação governante é:

$$\frac{\partial T}{\partial t} + u\frac{\partial T}{\partial x} = K\frac{\partial^2 T}{\partial x^2}$$

onde:

- $T(x, t)$ é a temperatura,
- u é a velocidade de advecção,
- K é o coeficiente de difusão numérico.

Esquema numérico (diferenças finitas explícitas)

Usamos discretização de primeira ordem no tempo e segunda ordem no espaço:

$$T_i^{n+1} = T_i^n - \frac{u\Delta t}{2\Delta x}(T_{i+1}^n - T_{i-1}^n) + \frac{K\Delta t}{(\Delta x)^2}(T_{i+1}^n - 2T_i^n + T_{i-1}^n)$$

Observações e assimilação 3D-Var

Vamos supor:

$$J(x) = \left(x - x_b\right)^T B^{-1}(x - x_b) + \left(y - Hx\right)^T R^{-1}(y - Hx)$$

- O modelo fornece o campo de fundo (background) $x_b = T^b$,
- Temos observações esparsas de temperatura y,
- O operador de observação H extrai os pontos observados, H
- O 3D-Var minimiza

A solução analítica é:

$$x_a = x_b + BH^T\left(HBH^T + R\right)^{-1}(y - Hx_b)$$

Observações e assimilação 3D-Var

Vamos supor:

```
program advec_diff_3dvar
  implicit none
  integer, parameter :: nx = 50, nt = 50
  real(8), parameter :: L = 1.0d0, dt = 0.001d0, u = 1.0d0, K = 0.01d0
  real(8) :: dx, t
  real(8), dimension(nx) :: T, Tnew, Tb, Ta, y, diff
  real(8), dimension(nx,nx) :: B, Ht, H, R, I, tmp, gain
  integer :: i, n

  dx = L / (nx-1)

  ! --- Condição inicial (gaussiana) ---
  do i = 1, nx
    T(i) = exp(-((i*dx - 0.5d0)**2)/0.01d0)
  end do
  Tb = T

  ! --- Matriz de covariância B (identidade simplificada) ---
  B = 0.0d0
  do i = 1, nx
    B(i,i) = 0.05d0
  end do

  ! --- Matriz de erro de observação R ---
  R = 0.0d0
  R(1,1) = 0.02d0

  ! --- Operador de observação H (observa apenas 1 ponto central) ---
  H = 0.0d0
  H(1, nx/2) = 1.0d0
  Ht = transpose(H)

  ! --- Simulação do modelo (background) ---
  do n = 1, nt
    call advec_diff_step(T, Tnew, u, K, dx, dt, nx)
    T = Tnew
  end do
  Tb = T   ! estado background

  ! --- Observação sintética ---
  y = 0.0d0
  y(1) = Tb(nx/2) + 0.1d0   ! adiciona erro positivo

  ! --- Aplicação do 3DVar ---
  tmp = matmul(H, matmul(B, Ht)) + R
  gain = matmul(B, matmul(Ht, inv1x1(tmp)))   ! ganho de Kalman
  Ta = Tb + matmul(gain, (y - matmul(H, Tb)))

  ! --- Resultado ---
  print *, "Ponto observado:", nx/2
  print *, "Background:", Tb(nx/2)
  print *, "Observação:", y(1)
  print *, "Análise:", Ta(nx/2)
contains
  subroutine advec_diff_step(T, Tnew, u, K, dx, dt, nx)
    implicit none
    integer, intent(in) :: nx
    real(8), intent(in) :: u, K, dx, dt
    real(8), dimension(nx), intent(in) :: T
    real(8), dimension(nx), intent(out) :: Tnew
    integer :: i
    Tnew = T
    do i = 2, nx-1
      Tnew(i) = T(i) - (u*dt/(2.d0*dx))*(T(i+1)-T(i-1)) + (K*dt/(dx*dx))*(T(i+1)-2*T(i)+T(i-1))
    end do
    ! Condições de contorno periódicas
    Tnew(1) = T(nx-1)
    Tnew(nx) = T(2)
  end subroutine advec_diff_step

  function inv1x1(a) result(ainv)
    real(8), dimension(1,1), intent(in) :: a
    real(8), dimension(1,1) :: ainv
    ainv(1,1) = 1.d0 / a(1,1)
  end function inv1x1

end program advec_diff_3dvar
```

Paulo Yoshio Kubota

Como esse programa funciona:

1.Modelo físico: propaga uma distribuição de temperatura gaussiana com o modelo de advecção e difusão;

2.Observação: coleta um único ponto (no centro da malha) com erro;

3.3DVar: ajusta o campo de temperatura usando a observação — suavemente, devido à covariância B;

4.Saída: mostra o valor antes, observado e analisado.

Expansão para 4D-Var

No **4D-Var**, o custo considera a evolução temporal:

$$J(x_0) = (x_0 - x_b)^T B^{-1}(x_0 - x_b) + \sum_k \left(y_k - H\boldsymbol{M_{0,k}}(x_0)\right)^T (R_k)^{-1}\left(y_k - H\boldsymbol{M_{0.k}}(x_0)\right)$$

onde $\boldsymbol{M_{0,k}}$ é o modelo propagador.

A implementação requer **integração direta e adjunta** (ou iteração no tempo para minimizar J), o que pode ser feito com um gradiente descendente.

O programa em **Fortran completo do 4D-Var** para o mesmo modelo advecção–difusão (usando um esquema iterativo para minimizar J)

Expansão para 4D-Var

No **4D-Var**, o custo considera a evolução temporal:

$$J(x_0) = (x_0 - x_b)^T B^{-1}(x_0 - x_b) + \sum_k \left(y_k - HM_{0,k}(x_0)\right)^T (R_k)^{-1}\left(y_k - HM_{0.k}(x_0)\right)$$

Descrição resumida do que o programa faz:

- monta o **operador linear do modelo** A (passo explícito advecção+difusão);
- gera uma **"verdade"** (trajetória) y_k a partir de uma condição inicial (duas gaussianas);
- gera **observações ruidosas** (H seleciona pontos);
- divide o período total em **janelas de assimilação** de comprimento win;
- para cada janela resolve **as equações normais do 4D-Var** na variável inicial x_0 da janela:

1. Função custo em forma matricial

Escrevamos a função custo 4D-Var (discreta no tempo) como função de $\mathbf{x}_0$:

$$J(x_0) = \frac{1}{2}(x_0 - x_b)^T B^{-1}(x_0 - x_b) + \frac{1}{2}\sum_{i=1}^{N}[y_i - H_i M_i x_0]^T R_i^{-1}[y_i - H_i M_i x_0]$$

onde:

- $\mathbf{x}_0$:estado inicial (variável de controle);
- $\mathbf{x}_b$:*background* no tempo inicial;
- $\mathbf{B}$: matriz de covariância do erro de background;
- $\mathbf{y}_i$:vetor de observações no tempo t_i;
- $\mathbf{R}_i$:matriz de covariância do erro de observação em t_i;
- $\mathbf{M}_i$:propagador do modelo de t_0a t_i;
- $\mathbf{H}_i$:operador (linearizado) de observação em t_i.

2. Derivada de cada termo em relação a $\mathbf{x}_0$

Queremos minimizar $J(\mathbf{x}_0)$ .Para isso, calculamos o gradiente $\nabla J(\mathbf{x}_0)$e impomos:

$$\nabla J(x_0) = 0$$

2.1. Termo de background

Considere:

$$J_b(x_0) = \frac{1}{2}(x_0 - x_b)^T B^{-1}(x_0 - x_b)$$

2. Derivada de cada termo em relação a $\mathbf{x}_0$

Essa é uma forma quadrática padrão. Um resultado básico de cálculo matricial:

$$\frac{\partial}{\partial x}\left(\frac{1}{2}[x^T Ax]\right) = \frac{\partial}{\partial x}\left(\frac{1}{2}\right)([x^T Ax]) + \left(\frac{1}{2}A\right)\frac{\partial([x^T])}{\partial x}x + \left(\frac{1}{2}A\right)\frac{\partial x}{\partial x}([x^T])$$

$$\frac{\partial}{\partial x}\left(\frac{1}{2}[x^T Ax]\right) = +\left(\frac{1}{2}\right)\frac{\partial([x^T])}{\partial x}Ax + \left(\frac{1}{2}\right)\frac{\partial x}{\partial x}A([x^T])$$

Se x é simétrica $x = x^T$,então:

$$\frac{\partial}{\partial x}\left(\frac{1}{2}[x^T Ax]\right) = +\left(\frac{1}{2}\right)Ax + \left(\frac{1}{2}\right)A([x^T])$$

$$\frac{\partial}{\partial x}\left(\frac{1}{2}[x^T Ax]\right) = +\left(\frac{1}{2}\right)Ax + \left(\frac{1}{2}\right)Ax$$

$$\frac{\partial}{\partial x}\left(\frac{1}{2}[x^T Ax]\right) = Ax$$

2. Derivada de cada termo em relação a $\mathbf{x}_0$

No nosso caso, fazemos a mudança de variável $\mathbf{z} = \mathbf{x}_0 - \mathbf{x}_b$:

$$J_b = \frac{1}{2} z^T B^{-1} z \qquad , onde\ z = (x_0 - x_b)$$

Então usando a definição do calculo matricial:

$$\frac{\partial J_b}{\partial z} = B^{-1} z = B^{-1}(x_0 - x_b)$$

Como $\mathbf{z}$ depende linearmente de $\mathbf{x}_0$,($\mathbf{z} = \mathbf{x}_0 - \mathbf{x}_b$:) temos:

$$\nabla_{x_0} J_b(x_0) = B^{-1}(x_0 - x_b)\text{c}$$

2.2. Termo de observações em um tempo t_i

Agora olhamos para um termo genérico do somatório:

$$J_{0,i}(x_0) = \frac{1}{2}[y_i - H_i M_i x_0]^T R_i^{-1}[y_i - H_i M_i x_0]$$

Definindo:

$$d_i(x_0) = y_i - H_i M_i x_0$$

Então:

$$J_{0,i}(x_0) = \frac{1}{2}[d_i]^T R_i^{-1}[d_i]$$

2.2. Termo de observações em um tempo t_i

Calculamos o gradiente em duas etapas:

Derivada de $J_{o,i}$em relação a $\mathbf{d}_i$:

$$J_{0,i}(x_0) = \frac{1}{2}[d_i]^T R_i^{-1}[d_i]$$

$$\frac{\partial J_{0,i}(d_i)}{\partial d_i} = \frac{1}{2}[d_i]^T R_i^{-1}[d_i]$$

$$\frac{\partial J_{0,i}(d_i)}{\partial d_i} = \frac{\partial\left(\frac{1}{2}\right)}{\partial d_i}[d_i]^T R_i^{-1}[d_i] + \left(\frac{1}{2}\right)\frac{\partial [d_i]^T}{\partial d_i} R_i^{-1}[d_i] + \left(\frac{1}{2}\right)[d_i]^T \frac{\partial R_i^{-1}}{\partial d_i}[d_i] + \left(\frac{1}{2}\right)[d_i]^T R_i^{-1}\frac{\partial [d_i]}{\partial d_i}$$

Se x é simétrica $d = d^T$,então:

Então usando a definição do calculo matricial:

$$\frac{\partial J_{0,i}(d_i)}{\partial d_i} = R_i^{-1}[d_i]$$

Derivada de $\mathbf{d}_i$ em relação a $\mathbf{x}_0$:

$$d_i(x_0) = y_i - H_i M_i x_0$$

$$\frac{\partial d_i(x_0)}{\partial x_0} = \frac{\partial y_i}{\partial x_0} - \frac{\partial [H_i M_i x_0]}{\partial x_0} = -\frac{\partial [H_i]}{\partial x_0}[M_i x_0] - [H_i]\frac{\partial [M_i]}{\partial x_0}[x_0] - [H_i M_i]\frac{\partial [x_0]}{\partial x_0}$$

$$\frac{\partial d_i(x_0)}{\partial x_0} = -[H_i M_i]$$

Usando a regra da cadeia (em notação de gradiente: transpostas entram):

$$\frac{\partial J_{0,i}(d_i)}{\partial d_i} = \frac{\partial \left(d_i(x_0)\right)}{\partial x_0}\frac{\partial J_{0,i}}{\partial d_i}$$

$$\frac{\partial J_{0,i}(d_i)}{\partial d_i} = \left[\frac{\partial \left(d_i(x_0)\right)}{\partial x_0}\right]^T \frac{\partial J_{0,i}}{\partial d_i} = (-[H_i M_i]^T)\left([R_i^{-1} d_i]\right) = -[H_i M_i]^T [R_i^{-1} d_i]$$

$$\frac{\partial J_{0,i}(d_i)}{\partial d_i} = \left[\frac{\partial \left(d_i(x_0)\right)}{\partial x_0}\right]^T \frac{\partial J_{0,i}}{\partial d_i} = -[H_i M_i]^T [R_i^{-1} d_i]$$

2.2. Termo de observações em um tempo t_i

$$\frac{\partial J_{0,i}(d_i)}{\partial d_i} = \left[\frac{\partial\left(d_i(x_0)\right)}{\partial x_0}\right]^T \frac{\partial J_{0,i}}{\partial d_i} = -[H_i M_i]^T \left[R_i^{-1} d_i\right] \qquad d_i(x_0) = y_i - H_i M_i x_0$$

Logo:

$$\nabla_{x_0} J_b(x_0) = B^{-1}(x_0 - x_b) \qquad \nabla_{x_0} J_{o,i}(x_0) = -M_i^T H_i^T R_i^{-1}[y_i - H_i M_i x_0]$$

3. Gradiente total da função custo

$$\nabla_{x_0} J_b(x_0) = B^{-1}(x_0 - x_b)$$

$$\nabla_{x_0} J_{o,i}(x_0) = -M_i^T H_i^T R_i^{-1}[y_i - H_i M_i x_0]$$

Somando o termo de *background* e todos os termos de observação:

$$\nabla_{x_0} J(x_0) = \nabla_{x_0} J_b(x_0) + \nabla_{x_0} J_{o,i}(x_0)$$

$$\nabla_{x_0} J(x_0) = B^{-1}(x_0 - x_b) - \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1}[y_i - H_i M_i x_0]$$

Para minimizar, impomos: $\nabla_{x_0} J(x_0) = 0$

$$B^{-1}(x_0 - x_b) - \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1}[y_i - H_i M_i x_0] = 0$$

4. Rearranjando para obter $\mathbf{x}_0$

$$B^{-1}(x_0 - x_b) - \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1}[y_i - H_i M_i x_0] = 0$$

Vamos desenvolver o segundo termo:

$$y_i - H_i M_i x_0 \Rightarrow M_i^T H_i^T R_i^{-1}[y_i - H_i M_i x_0] = M_i^T H_i^T R_i^{-1} y_i - M_i^T H_i^T R_i^{-1} H_i M_i x_0$$

Substituindo na equação do gradiente nulo:

$$B^{-1}(x_0 - x_b) - \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} y_i - M_i^T H_i^T R_i^{-1} H_i M_i x_0 = 0$$

$$B^{-1}(x_0 - x_b) - \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} y_i + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} H_i M_i x_0 = 0$$

2.2. Termo de observações em um tempo t_i

$$B^{-1}(x_0 - x_b) - \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} y_i + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} H_i M_i x_0 = 0$$

Agora agrupamos os termos em $\mathbf{x}_0$de um lado, e os termos independentes de $\mathbf{x}_0$do outro.

$$B^{-1}(x_0) - B^{-1}(x_b) - \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} y_i + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} H_i M_i x_0 = 0$$

$$B^{-1}(x_0) + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} H_i M_i x_0 = B^{-1}(x_b) + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} y_i$$

Agrupe os termos com $\mathbf{x}_0$:

$$\left[B^{-1} + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} H_i M_i \right] x_0 = \left[B^{-1} x_b + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} y_i \right]$$

$$\left[B^{-1} + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} H_i M_i\right] x_0 = \left[B^{-1} x_b + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} y_i\right]$$

Agora está na forma de um sistema linear:

$$H_j x_0 = g$$

onde:

•a **"Hessiana"** (aqui, a matriz de segunda derivada) é:

$$H_j = \left[B^{-1} + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} H_i M_i\right]$$

e o lado direito é:

$$g = \left[B^{-1} x_b + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} y_i\right]$$

2.2. Termo de observações em um tempo t_i

Assumindo que $\mathbf{B}$ e $\mathbf{R}_i$ são definidas positivas, $\mathbf{H}_J$é definida positiva ⇒ J é estritamente convexa ⇒ existe um único mínimo.

Logo, **a solução analítica** é:

$$\left[B^{-1} + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} H_i M_i\right] x_0 = \left[B^{-1} x_b + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} y_i\right]$$

$$x_0^a = x_0 = \left[B^{-1} + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} H_i M_i\right]^{-1} \left[B^{-1} x_b + \sum_{i=1}^{N} M_i^T H_i^T R_i^{-1} y_i\right]$$

Esse é o $\mathbf{x}_0$que minimiza $J(\mathbf{x}_0)$no caso linear.

6. Comentário rápido sobre o caso não linear

Para o caso **não linear** (**modelo e operador de observação não lineares**), a forma de J é a mesma, mas:

•$\mathbf{M}_i$ e $\mathbf{H}_i$ são substituídos pelos operadores não lineares $\mathcal{M}_i, \mathcal{H}_i$;

•a minimização é feita iterativamente (Gauss–Newton, quasi-Newton, etc.);

•a cada iteração, linearizam-se $\mathcal{M}_i$ e $\mathcal{H}_i$ em torno de uma trajetória, e aplica-se exatamente a álgebra que fizemos acima para achar o incremento $\delta\mathbf{x}_0$.

Expansão para 4D-Var

No **4D-Var**, o custo considera a evolução temporal:

$$J(x_0) = \frac{1}{2}(x_0 - x_b)^T B^{-1}(x_0 - x_b) + \frac{1}{2}\sum_{i=1}^{N}[y_i - H_i M_i x_0]^T R_i^{-1}[y_i - H_i M_i x_0]$$

$$\left[B^{-1} + \sum_{i=1}^{win} M_i^T H_i^T R_i^{-1} H_i M_i\right] x_0 = \left[B^{-1} x_b + \sum_{i=1}^{win} M_i^T H_i^T R_i^{-1} y_i\right]$$

- usando **Gradiente Conjugado (CG)** para evitar inversões diretas;
- propaga a solução ótima x0 pelo modelo na janela e grava a trajetória analisada;
- salva arquivos de saída para visualização

O código tem comentários explicativos e é dimensionado para que você possa alterar nx, nt, win, Ne, etc.

Salve como advec_diff_4dvar.f90 e compile:

```
gfortran -O2 advec_diff_4dvar.f90 -o advec_diff_4dvar
./advec_diff_4dvar
```

Após rodar, os arquivos gerados serão:

•4dvar_out.dat — cada linha: tempo, verdade (nx valores), background (nx), análise (nx)

•rmse_4dvar.dat — colunas: tempo, rmse_background, rmse_analysis

Expansão para 4D-Var

$$J(x_0) = (x_0 - x_b)^T B^{-1}(x_0 - x_b) + \sum_k \left(y_k - HM_{0,k}(x_0)\right)^T (R_k)^{-1}\left(y_k - HM_{0.k}(x_0)\right)$$

```
! advec_diff_4dvar.f90
program advec_diff_4dvar
  implicit none
  integer, parameter :: dp = selected_real_kind(15,307)
  integer, parameter :: nx = 60          ! grid points
  integer, parameter :: nt = 120         ! total time steps
  integer, parameter :: win = 12         ! window length (steps)
  real(dp), parameter :: L = 1.0_dp
  real(dp), parameter :: u = 1.0_dp
  real(dp), parameter :: kappa = 0.01_dp
  real(dp), parameter :: dx = L / real(nx-1, dp)
  real(dp), parameter :: dt = 0.25_dp * dx   ! choose stable dt
  real(dp), parameter :: sigma_b = 0.10_dp
  real(dp), parameter :: sigma_obs = 0.02_dp

  ! Observations: indices where observations exist (m per time)
  integer, parameter :: m = 6
  integer, parameter :: obs_idx(m) = (/ 6, 14, 22, 30, 38, 46 /)

  ! arrays
  real(dp), allocatable :: A(:,:), Ap(:,:,:), AT(:,:), ATp(:,:,:)
  real(dp), allocatable :: truth(:,:), bkg(:,:), analysis(:,:)
  real(dp), allocatable :: x0_true(:), x0_b(:)
  real(dp), allocatable :: y(:,:)
  real(dp), allocatable :: B(:,:), Binv(:,:), R(:,:), Rinv(:,:)
  real(dp), allocatable :: H(:,:), Ht(:,:)

  ! temp / CG matrices
  real(dp), allocatable :: LHS(:,:), RHS(:), x0_opt(:)

  integer :: i,j,k,t,win_k,info
  integer :: outunit, rmseunit
  real(dp) :: t0

  call seed_rng(123456)
```

$$\left(B^{-1} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} H A^k\right) x_0 = B^{-1} x_{b0} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} y_k$$

Paulo Yoshio Kubota

Expansão para 4D-Var

$$J(x_0) = (x_0 - x_b)^T B^{-1}(x_0 - x_b) + \sum_k \left(y_k - HM_{0,k}(x_0)\right)^T (R_k)^{-1}\left(y_k - HM_{0.k}(x_0)\right)$$

```
! allocate
allocate(A(nx,nx), AT(nx,nx))
allocate(Ap(nx,nx,win+1), ATp(nx,nx,win+1))
allocate(truth(nt+1,nx), bkg(nt+1,nx), analysis(nt+1,nx))
allocate(x0_true(nx), x0_b(nx))
allocate(y(nt,m))
allocate(B(nx,nx), Binv(nx,nx), R(m,m), Rinv(m,m))
allocate(H(m,nx), Ht(nx,m))
allocate(LHS(nx,nx), RHS(nx), x0_opt(nx))

! build A
call build_A(A, nx, dx, dt, u, kappa)
AT = transpose(A)

! build powers A^k and (A^T)^k

Ap(:,:,1) = identity(nx)
ATp(:,:,1) = identity(nx)
do k = 2, win+1
  Ap(:,:,k)  = matmul(A, Ap(:,:,k-1))
  ATp(:,:,k) = transpose(Ap(:,:,k))
end do

! truth initial condition (two gaussian bumps)
call make_truth_ic(nx, dx, x0_true)
truth(1,:) = x0_true
do t = 1, nt
  truth(t+1,:) = matmul(A, truth(t,:))
end do

! observations (one per time step at obs indices)
do t = 1, nt
  do i = 1, m
    y(t,i) = truth(t+1, obs_idx(i)) + sigma_obs * randn()
  end do
end do
```

$$\left(B^{-1} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} H A^k\right) x_0 = B^{-1} x_{b0} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} y_k$$

Paulo Yoshio Kubota

Expansão para 4D-Var

$$J(x_0) = (x_0 - x_b)^T B^{-1}(x_0 - x_b) + \sum_k \left(y_k - HM_{0,k}(x_0)\right)^T (R_k)^{-1}\left(y_k - HM_{0.k}(x_0)\right)$$

```
! background: perturb initial truth and propagate free
x0_b = x0_true + sigma_b * randn_vec(nx)
bkg(1,:) = x0_b
do t = 1, nt
  bkg(t+1,:) = matmul(A, bkg(t,:))
end do

! build B (diagonal) and Binv
B = 0.0_dp
do i = 1, nx
  B(i,i) = sigma_b**2
  Binv(i,i) = 1.0_dp / B(i,i)
end do

! build R and Rinv (m x m)
R = 0.0_dp
do i = 1, m
  R(i,i) = sigma_obs**2
  Rinv(i,i) = 1.0_dp / R(i,i)
end do

! build H (m x nx) and Ht (nx x m)
H = 0.0_dp
do i = 1, m
  H(i, obs_idx(i)) = 1.0_dp
end do
Ht = transpose(H)

! open output files
open(newunit=outunit, file="4dvar_out.dat", status="replace", action="write")
write(outunit, '(A)') '# t truth(...) background(...) analysis(...)'
open(newunit=rmseunit, file="rmse_4dvar.dat", status="replace", action="write")
write(rmseunit, '(A)') '# t rmse_background rmse_analysis'
```

$$\left(B^{-1} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} H A^k\right) x_0 = B^{-1} x_{b0} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} y_k$$

Paulo Yoshio Kubota

Expansão para 4D-Var

$$J(x_0) = (x_0 - x_b)^T B^{-1}(x_0 - x_b) + \sum_k \left(y_k - HM_{0,k}(x_0)\right)^T (R_k)^{-1}\left(y_k - HM_{0.k}(x_0)\right)$$

```
! initialize analysis with background initial guess
analysis(1,:) = bkg(1,:)

t = 1
do while (t <= nt)
  ! Determine window end
  win_k = min(win, nt - t + 1)

  ! Build LHS and RHS for window starting at time t
  ! LHS = Binv + sum_{k=1..win_k} AT^k * H^T * Rinv * H * A^k
  LHS = Binv
  do k = 1, win_k
    LHS = LHS + matmul( ATp(:,:,k+1), matmul( matmul(Ht, Rinv), matmul(H, Ap(:,:,k+1)) ) )
  end do

  ! RHS = Binv * xb0 + sum_k AT^k * H^T * Rinv * y_k
  RHS = 0.0_dp
  RHS = matmul(Binv, bkg(t,:))   ! xb0 = background at window start
  do k = 1, win_k
    RHS = RHS + matmul( ATp(:,:,k+1), matmul(Ht, matmul(Rinv, y(t + k - 1, :))) )
  end do

  ! Solve LHS * x0_opt = RHS using CG (x0 initial guess = xb0)
  x0_opt = bkg(t,:)    ! initial guess
  call cg_solve(LHS, RHS, x0_opt, 1000, 1.0e-9_dp)

  ! propagate optimal x0 through the window and store analysis
  do k = 0, win_k-1
    analysis(t + k, :) = matmul(Ap(:,:,k+1), x0_opt)
  end do
  ! also write the last time of window
  analysis(t + win_k, :) = matmul(Ap(:,:,win_k+1), x0_opt)

  ! advance by window length (no overlap here, could do overlap if desired)
  t = t + win_k
end do
```

$$\left(B^{-1} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} H A^k\right) x_0 = B^{-1} x_{b0} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} y_k$$

Paulo Yoshio Kubota

Expansão para 4D-Var

$$J(x_0) = (x_0 - x_b)^T B^{-1}(x_0 - x_b) + \sum_k \left(y_k - HM_{0,k}(x_0)\right)^T (R_k)^{-1}\left(y_k - HM_{0.k}(x_0)\right)$$

```
! compute rmse per time and write outputs
do t = 1, nt+1
  write(outunit, '(I6,1X,60F12.6,1X,60F12.6,1X,60F12.6)') t-1, truth(t,:), bkg(t,:), analysis(t,:)
  write(rmseunit, '(I6,2F12.6)') t-1, rmse_vec(bkg(t,:), truth(t,:)), rmse_vec(analysis(t,:), truth(t,:))
end do

close(outunit)
close(rmseunit)
print *, '4D-Var run finished. Output: 4dvar_out.dat, rmse_4dvar.dat'

contains

!============================================================
subroutine build_A(A, nx, dx, dt, u, kappa)
  integer, intent(in) :: nx
  real(dp), intent(in) :: dx, dt, u, kappa
  real(dp), intent(out) :: A(nx,nx)
  integer :: i
  A = 0.0_dp
  A(1,1) = 1.0_dp
  A(nx,nx) = 1.0_dp
  do i = 2, nx-1
    A(i,i-1) = u * dt / dx + kappa * dt / (dx*dx)
    A(i,i)   = 1.0_dp - u * dt / dx - 2.0_dp * kappa * dt / (dx*dx)
    A(i,i+1) = kappa * dt / (dx*dx)
  end do
end subroutine build_A
```

$$\left(B^{-1} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} H A^k\right) x_0 = B^{-1} x_{b0} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} y_k$$

Paulo Yoshio Kubota

Expansão para 4D-Var

$$J(x_0) = (x_0 - x_b)^T B^{-1}(x_0 - x_b) + \sum_k \left(y_k - HM_{0,k}(x_0)\right)^T (R_k)^{-1}\left(y_k - HM_{0.k}(x_0)\right)$$

```
subroutine make_truth_ic(nx, dx, x0)
  integer, intent(in) :: nx
  real(dp), intent(in) :: dx
  real(dp), intent(out) :: x0(nx)
  integer :: i
  do i = 1, nx
    x0(i) = exp( - (( (real(i-1,dp)*dx) - 0.25_dp )**2) / (2.0_dp*(0.05_dp**2)) ) + &
          0.5_dp * exp( - (( (real(i-1,dp)*dx) - 0.65_dp )**2) / (2.0_dp*(0.08_dp**2)) )
  end do
end subroutine make_truth_ic

!=============================================================
function identity(n) result(I)
  integer, intent(in) :: n
  real(dp) :: I(n,n)
  integer :: i
  I = 0.0_dp
  do i = 1, n
    I(i,i) = 1.0_dp
  end do
end function identity

!=============================================================
function randn() result(r)
  real(dp) :: r
  real(dp) :: u1,u2
  call random_number(u1); call random_number(u2)
  if (u1 < 1.0e-12_dp) u1 = 1.0e-12_dp
  r = sqrt(-2.0_dp*log(u1)) * cos(2.0_dp*acos(-1.0_dp)*u2)
end function randn
```

$$\left(B^{-1} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} H A^k\right) x_0 = B^{-1} x_{b0} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} y_k$$

Paulo Yoshio Kubota

Expansão para 4D-Var

$$J(x_0) = (x_0 - x_b)^T B^{-1}(x_0 - x_b) + \sum_k \left(y_k - HM_{0,k}(x_0)\right)^T (R_k)^{-1}\left(y_k - HM_{0.k}(x_0)\right)$$

```
!=============================================================
function randn_vec(n) result(v)
  integer, intent(in) :: n
  real(dp) :: v(n)
  integer :: i
  do i = 1, n
    v(i) = randn()
  end do
end function randn_vec
```

$$\left(B^{-1} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} H A^k\right) x_0 = B^{-1} x_{b0} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} y_k$$

```
!=============================================================
 subroutine cg_solve(A, b, x, maxit, tol)
  ! Conjugate Gradients for symmetric positive-definite A
  real(dp), intent(in) :: A(:,:), b(:)
  real(dp), intent(inout) :: x(:)
  integer, intent(in) :: maxit
  real(dp), intent(in) :: tol
  integer :: n, i, it
  real(dp), allocatable :: r(:), p(:), Ap(:)
  real(dp) :: alpha, beta, rsold, rsnew, pAp

  n = size(b)
  allocate(r(n), p(n), Ap(n))
  r = b - matmul(A, x)
  p = r
  rsold = dot_product(r,r)
  do it = 1, maxit
    Ap = matmul(A, p)
    pAp = dot_product(p, Ap)
    if (abs(pAp) < 1.0e-30_dp) exit
    alpha = rsold / pAp
    x = x + alpha * p
    r = r - alpha * Ap
    rsnew = dot_product(r,r)
    if (sqrt(rsnew) < tol) exit
    beta = rsnew / rsold
    p = r + beta * p
    rsold = rsnew
  end do
  deallocate(r, p, Ap)
end subroutine cg_solve
```



Expansão para 4D-Var

$$J(x_0) = (x_0 - x_b)^T B^{-1}(x_0 - x_b) + \sum_k \left(y_k - HM_{0,k}(x_0)\right)^T (R_k)^{-1}\left(y_k - HM_{0.k}(x_0)\right)$$



```
  !=============================================================
 function rmse_vec(a, b) result(rmse)
   real(dp), intent(in) :: a(:), b(:)
   real(dp) :: rmse
   integer :: n
   n = size(a)
   rmse = sqrt(sum((a - b)**2) / real(n, dp))
 end function rmse_vec

end program advec_diff_4dvar
```

$$\left(B^{-1} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} H A^k\right) x_0 = B^{-1} x_{b0} + \sum_{k=1}^{win} A^{Tk} H^T R^{-1} y_k$$

$$J(x_0) = (x_0 - x_b)^T B^{-1}(x_0 - x_b) + \sum_k \left(y_k - HM_{0,k}(x_0)\right)^T (R_k)^{-1}\left(y_k - HM_{0.k}(x_0)\right)$$

Observações finais e sugestões

•O programa usa um **LHS explicitamente montado** (matriz densa nx x nx). Para nx grande, troque a montagem por um operador que aplique LHS a um vetor (matvec) sem formar LHS inteiro — isso permite CG escalável.

•H está implementado como seleção de índices (m pontos). Para observações esparsas no tempo, adapte y(t, :) adequadamente.

•A janela aqui é não-sobreposta (stride = win). Você pode escolher overlap para suavizar trocas entre janelas.

•Para **4D-Var não-lineal**, a técnica requer um modelo adjunto para calcular gradiente; aqui usamos a vantagem do **modelo linear** (A) para montar a normal diretamente.

Módulo 2 — Formulação Matemática

```
import numpy as np

# Dados do modelo (temperatura estimada)
x_b = np.array([10, 12, 14, 16, 18, 20])  # Temperatura no modelo

# Observação
y = np.array([15.5])  # Observação de temperatura

# Matriz de covariância do erro de modelo (B)
B = np.array([[1]])

# Matriz de covariância do erro de observação (R)
R = np.array([[0.5]])

# Operador de observação (H) - no caso, uma observação no ponto 2
H = np.array([[0, 0, 1, 0, 0, 0]])

# Cálculo da análise
Ht = H.T
B_inv = np.linalg.inv(B)
R_inv = np.linalg.inv(R)

# Fórmula da análise (ajuste do modelo)
x_hat = x_b + B_inv @ Ht @ np.linalg.inv(H @ B_inv @ Ht + R_inv) @ (y - H @ x_b)

print("Estado corrigido após assimilação:", x_hat)
```

Exercício 2:

1.Implemente o processo de assimilação utilizando a fórmula do **3D-Var** para um modelo simples de temperatura.

1.Varie os valores das matrizes de covariância **B** e **R** e observe como a análise $\hat{x}$ muda.

2.Tente aplicar esse processo para diferentes pontos de observação.

Conclusão do Módulo 2

•A formulação matemática da assimilação de dados envolve a definição de um vetor de estado e a relação com as observações, considerando os erros de modelo e de observação.

•A função custo é central para os métodos de assimilação, como o **3D-Var**, onde buscamos minimizar a discrepância entre o modelo e as observações.

•Com a fórmula de análise, podemos ajustar o estado do modelo com base nas observações e suas incertezas.